JAVA 2

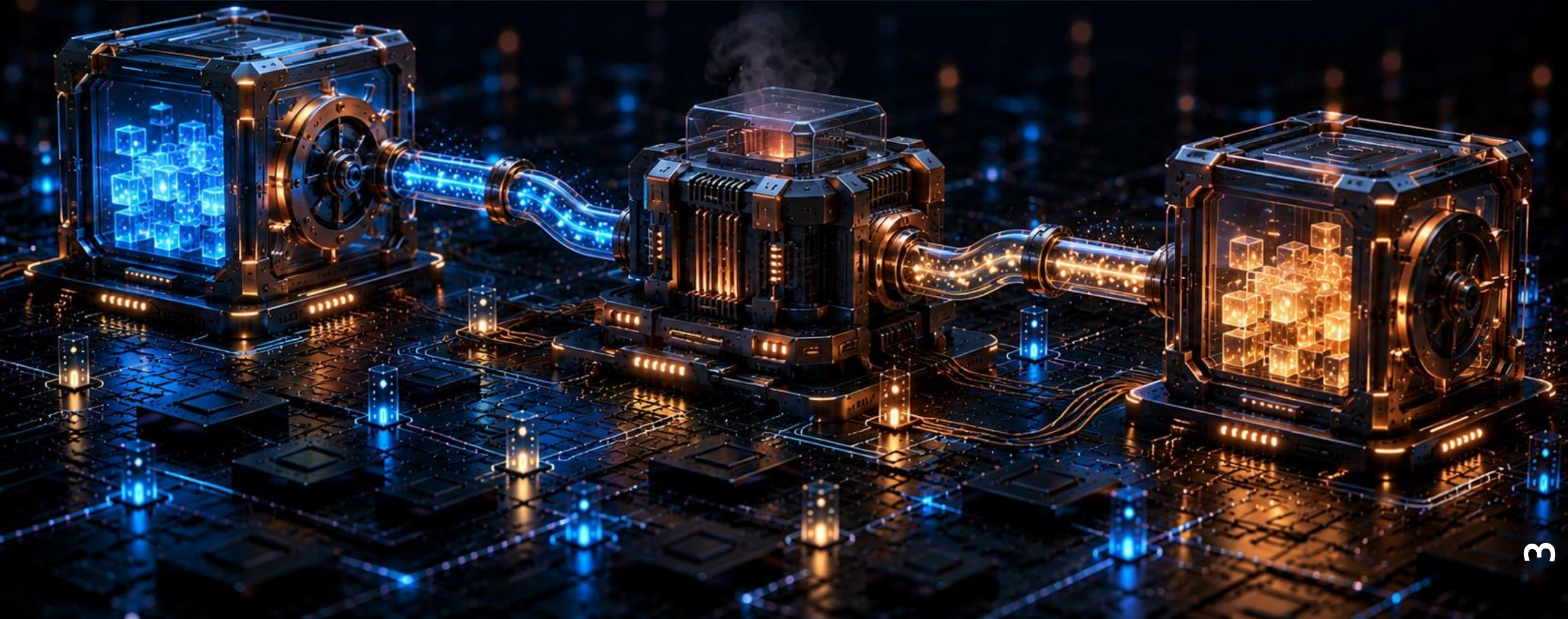
Module FIC

Fichiers

Au programme

1. Les flux
2. Fermer un flux
3. Le texte
4. Les objets

1. Ce qui entre, ce qui sort



Tout s'efface à l'arrêt

Un objet vit en mémoire, le temps que tourne le programme

Un fichier vit sur le disque, et survit à l'arrêt

Écrire un fichier, c'est envoyer une suite d'octets vers le disque

Reste à décider ce que représentent ces octets

Vous écrivez déjà dans un flux

Un flux est une suite de données qui va dans un seul sens

La console en ouvre deux pour vous : `System.out` et `System.in`

Un fichier s'ouvre de la même façon, et se manipule pareil

```
// System.out est un flux de sortie, ouvert pour vous
PrintStream console = System.out;
console.println("Uccle : 12 degres");

// System.in est un flux d'entree, lu par le Scanner
Scanner clavier = new Scanner(System.in);
String saisie = clavier.nextLine();
```

Deux familles de flux, quatre classes racines

Famille	Lire	Écrire
Octets, la racine	`InputStream`	`OutputStream`
Octets, vers un fichier	`FileInputStream`	`FileOutputStream`
Caractères, la racine	`Reader`	`Writer`
Caractères, vers un fichier	`FileReader`	`FileWriter`

Les flux s'emboîtent

Chaque classe se construit sur la précédente, et lui ajoute une capacité

La classe `FileOutputStream` envoie des données dans un fichier, octet par octet

Les suivantes regroupent les octets, puis traduisent les `int` en octets

```
OutputStream fichier = new FileOutputStream("temperatures.bin");
OutputStream tampon = new BufferedOutputStream(fichier);
DataOutputStream sortie = new DataOutputStream(tampon);

sortie.writeInt(12);          // quatre octets
sortie.writeDouble(12.5);     // huit octets
sortie.close();               // ferme les trois
```

Exercices

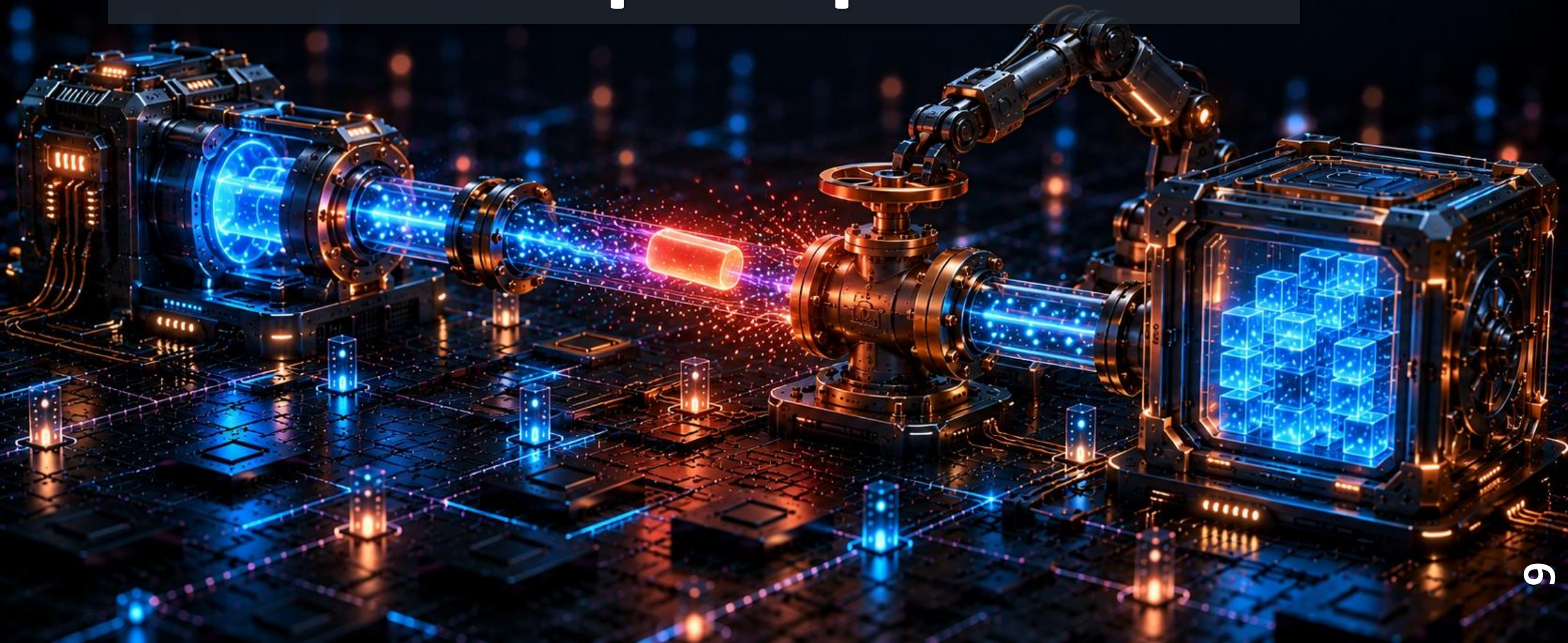


FIC.1 Trois classes qui se connaissent

fascicule FIC

Aucun fichier : le modèle des deux exercices suivants

2. Fermer quoi qu'il arrive



Un flux ouvert se ferme

Un flux ouvert retient le fichier, et le système limite le nombre de flux ouverts

Tant que le flux n'est pas fermé, son tampon peut ne pas être écrit

Le bloc `finally` garantit la fermeture, à ne jamais oublier

```
Writer sortie = new FileWriter("journal.txt");
try {
    sortie.write("Uccle;12");
} finally {
    sortie.close();     // meme si write echoue
}
```

Le `try` avec ressources

On peut déclarer la ressource (`FileWriter` par exemple) entre les parenthèses du `try`

Cette ressource sera fermée à la sortie du bloc, quoi qu'il arrive

Toute classe qui implémente `AutoCloseable` s'y prête, dont `Scanner`

```
try (Writer sortie = new FileWriter("journal.txt")) {
    sortie.write("Uccle;12");
}   // sortie.close() est appele ici, exception ou non
```

`IOException`, une exception contrôlée

Un fichier peut être absent, être verrouillé, ou le disque peut être plein

`IOException` est une exception contrôlée : elle doit être traitée ou déclarée

Ses filles précisent l'échec : `FileNotFoundException`, `EOFException`

```
public static String premiereLigne(File fichier) throws IOException {
    try (Scanner entree = new Scanner(fichier)) {
        return entree.nextLine();
    }
}
```

3. Écrire et relire du texte



Écrire comme sur la console

La classe `PrintWriter` offre `print`, `println` et `printf`, comme `System.out`

Elle se pose sur un `FileWriter`, qui atteint le fichier

À l'ouverture, le fichier est vidé ; avec `true`, on écrit à la suite

Constructor
`FileWriter(File file)`
`FileWriter(File file, boolean append)`

```
File fichier = new File("uccle.txt");
try (PrintWriter sortie = new PrintWriter(new FileWriter(fichier))) {
    sortie.println("Uccle;12");
    sortie.printf("%s;%d%n", "Ostende", 9);
}
```

Relire ligne par ligne

La classe `Scanner` lit un fichier comme elle lit le clavier

La méthode `hasNextLine` dit s'il reste une ligne, `nextLine` la retourne

La boucle suivante s'arrête d'elle-même à la fin du fichier

```
File fichier = new File("uccle.txt");
try (Scanner entree = new Scanner(fichier)) {
    while (entree.hasNextLine()) {
        System.out.println(entree.nextLine());
    }
}
```

Une ligne, plusieurs champs

Le texte ne garde que des caractères : à vous de choisir un séparateur

La méthode `split` découpe la ligne, `Integer.parseInt` reconvertit

L'écriture et la relecture suivent le même format, dans les deux sens

```java
String ligne = "Uccle;12";
String[] champs = ligne.split(";");

String station = champs[0];
int temperature = Integer.parseInt(champs[1]);
```

Un nom n'est pas un fichier

Un objet `File` désigne un chemin : le créer ne crée aucun fichier

Il permet d'interroger le système : `exists`, `length`, `mkdirs`

Les constructeurs de flux acceptent un `File` comme une chaîne

```
File dossier = new File("releves");
File fichier = new File(dossier, "uccle.txt");

dossier.mkdirs();                          // cree le dossier s'il manque
System.out.println(fichier.exists());
System.out.println(fichier.length());      // en octets
```

`Path` et `Files`

Depuis Java 7, l'interface `Path` désigne un chemin, comme `File`

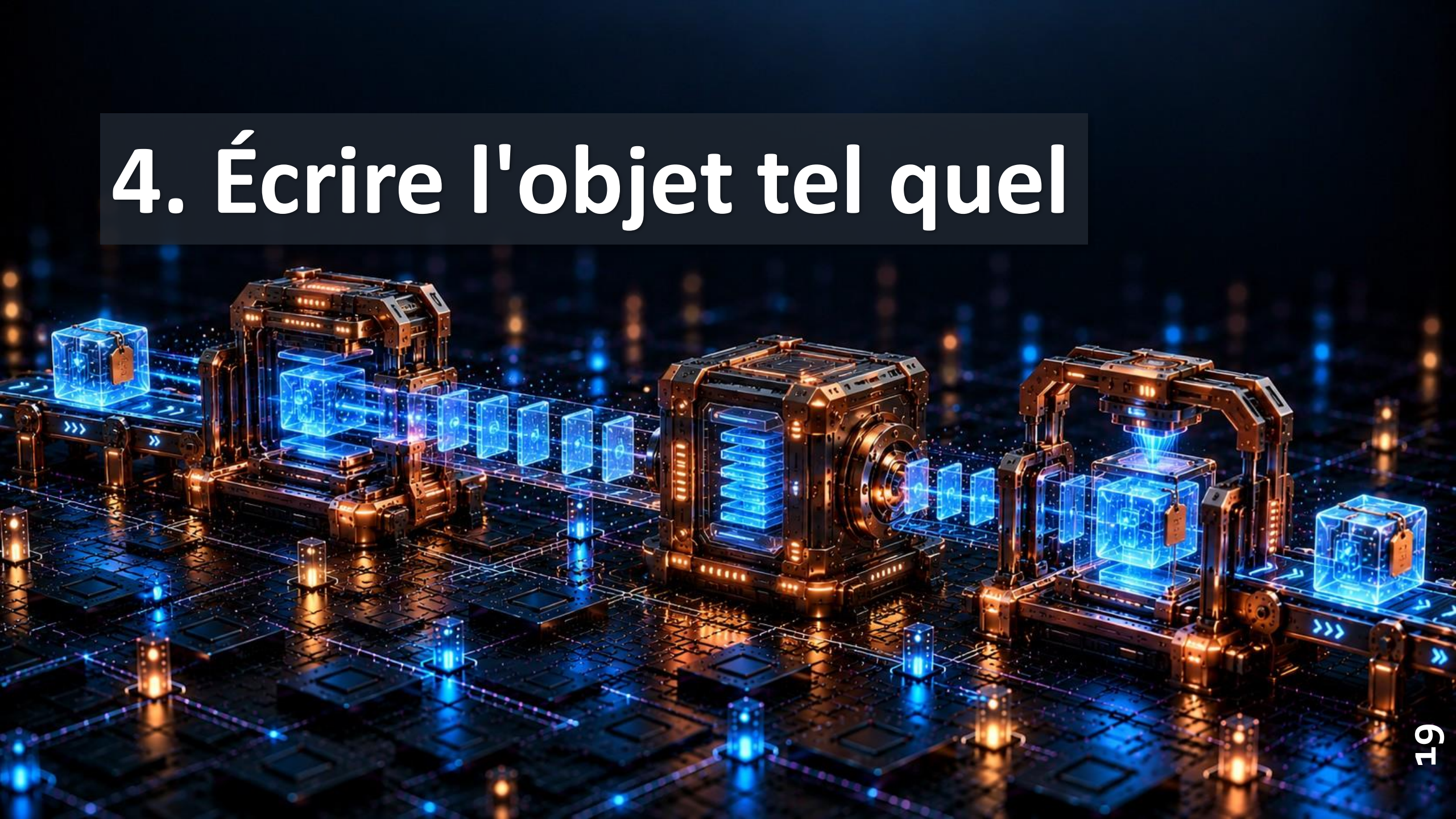
La classe `Files` regroupe des méthodes statiques prêtes à l'emploi

Elles ouvrent et ferment le fichier elles-mêmes

```
Path chemin = Path.of("releves", "uccle.txt");

Files.writeString(chemin, "Uccle;12\n");
List<String> lignes = Files.readAllLines(chemin);
```

4. Écrire l'objet tel quel



Un format à inventer pour chaque classe

Le texte demande, pour chaque classe, un format, une écriture, une relecture

Plus l'objet est riche, plus le format se complique

Java sait-il écrire l'objet lui-même ?

Démonstration



Démonstration

Les mêmes données, deux fois

Déroulé : Demo_FIC

Les flux d'objets

La classe `ObjectOutputStream` est une couche de plus, comme en 1.5

Elle ajoute `writeObject` aux méthodes des types primitifs

On relit dans l'ordre d'écriture, et on transtype

```
// ecrire : les methodes de DataOutputStream, plus une
sortie.writeInt(2026);
sortie.writeObject(releve);

// relire : dans le meme ordre
int annee = entree.readInt();
Releve lu = (Releve) entree.readObject();
```

Une étiquette, pas un contrat

L'interface `Serializable` ne déclare aucune méthode : c'est un marqueur

Rien n'est vérifié à la compilation : en cas de problème, l'écriture échoue à l'exécution

Le JDK l'a posée sur `String`, `Integer`, `ArrayList`, `LocalDate`…

```
public class Releve implements Serializable {
    private final String station;
    private final int temperature;

    // constructeur et accesseurs : rien d'autre a ajouter
}
```

Où s'arrête un fichier

Méthode de lecture	Comment s'annonce la fin
`Scanner.nextLine`	`hasNextLine()` retourne `false` avant
`BufferedReader.readLine`	la méthode retourne `null`
`DataInputStream.readInt`	elle lève une `EOFException`
`ObjectInputStream.readObject`	elle lève une `EOFException`

Exercices



FIC.2 Enregistrer des objets

fascicule FIC

FIC.3 Enregistrer du texte

fascicule FIC

Le fil rouge



G.35 Un journal qui survit

fil rouge GameHub

Chaque partie écrite dans un fichier, relue au lancement